

搜索，即 test-time compute

肖涵

Elastic 副总裁



微信



模型这边

榜单上挤满了人，第一名和第十名差几个千分点。

换谁家的 embedding，效果都差不多。

链路这边

召回、精排、混合检索、查询改写——该有的组件，十年前就都有了。

于是大家的结论

搜索是个成熟行业，剩下的都是工程活，值得投入的地方在别的方向上。

或者，坚持做训练，把训练做得更重。



搜索，本来就是一种 test-time compute。

这条轴上还有大量空间，而它跟模型规模无关。



test-time compute, 说的是同一笔算力花在哪一端。

给你一块 GPU,
你把它花在**训练时**,
还是**推理时**?

一句话：不去训一个更大的模型，而是让手上这个模型，在回答每个问题时多干一点活。训练阶段的钱已经花完了，这笔新的开销花在推理这一刻。

Best-of-N

采样多个候选答案，选出其中最好的一个。

Self-consistency

让模型把推理过程走多遍，取出现次数最多的答案。

Verifier search

用一个判分器在候选里搜索，投入的算力越多，结果越好。

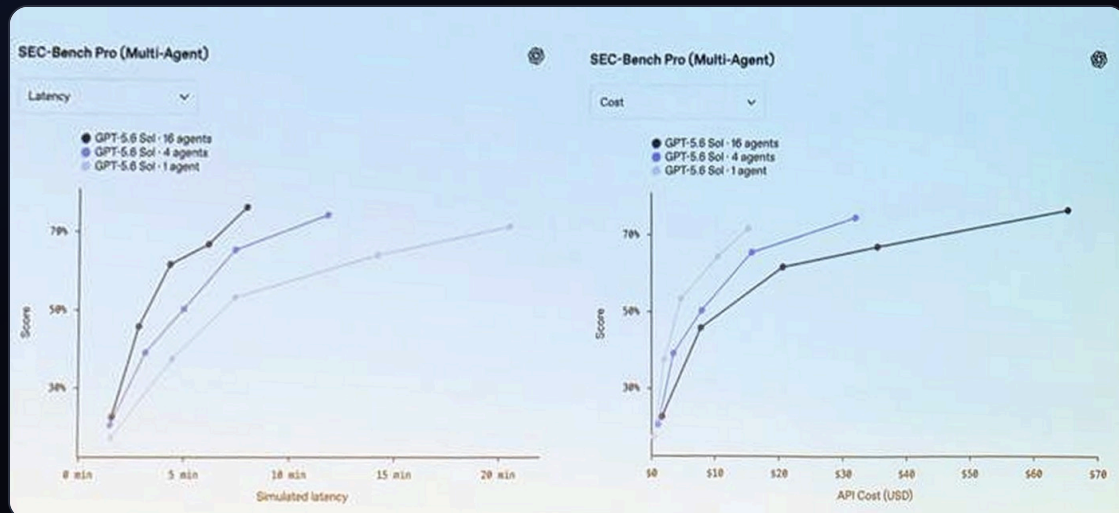
模型没变，权重一个没动。变的只是你愿意为这一次回答，多花多少算力。

让机器人在一手扑克上多想 **20 秒**，effect 相当于把模型放大 **10 万倍**、再多训 10 万倍的时间。

Noam Brown, OpenAI, 2024

20 秒，换 10 万倍。

这个兑换比例一旦成立，问题就从「模型够不够大」，变成了「你舍不舍得在推理时多花这一点」。



两年后的同一条曲线，OpenAI 在 SEC-Bench Pro 上把它重画了一遍：模型不变，只把 agent 从 1 个加到 16 个，分数随延迟和成本单调往上爬。



01 | 为什么说搜索就是 TTC

多跑几遍，跑得更深一点，然后看看能换回来什么

向量召回一遍，精排再来一遍，查询改写之后又是一遍，多路结果还要融合。这些环节没有一个是
在训练阶段完成的，**全都发生在用户按下回车之后**。这就是 test-time compute，只是我们一直管它
叫「搭链路」。

A

Multi-pass: 多跑几遍

同一个模型，换个角度再编码一次。把文档切成句子分别算，把图片切成局部分别看。**每一遍都会带回上一遍漏掉的东西。**

B

Deeper pass: 跑得更深

同一次前向，不只取最后那个池化向量，把中间的 patch、token 全都读出来用。**算力几乎没多花，信息多拿了一大截。**

C

Compose: 串起来

让 agent 自己决定先 grep 还是先 embed，要不要重排，要不要再搜一轮。**链路本身成了推理期的产物。**

单向量 COSINE



$$\cos(q, d)$$

一篇文档压缩成一个向量

最省算力的做法

句子级 MAXSIM



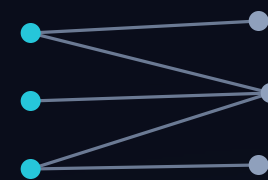
$$\max_s \cos(q, s)$$

还是那个模型，把文档切成句子各算

一遍

同一个模型，多算几遍

COLBERT 多向量



$$\sum_i \max_j \cos(q_i, d_j)$$

每个 query token 对全部 doc token

取 max

需要专门的多向量模型

中间这一列没有换模型，只是在推理时多算了几遍，效果却接近右边那种需要专门多向量模型的做法。



你花出去的

推理时间

买

multi-pass · deeper pass · 更长的 agent 循环

01 相关性

同一个任务，做得更准

02 新能力

模型压根没被训练过的事，它也能干

搜索的上限不取决于模型有多大，
而取决于你愿意为一次查询付出多少推理算力。

01

向量模型的 TTC

编码器冻住不动，只在推理期把同一批向量重新组合，域内 $\Delta nDCG@10$ 能爬 0.17。

换到的是 · 相关性

02

重排模型的 TTC

召回之后再上一段模型，让候选在同一个上下文里互相比。极端情况下，索引可以整个删掉。

换到的是 · 精度

03

用 TTC 换新能力

同样冻结的模型，这次不求它更准，而是让它做一件训练时根本不存在的任务。

换到的是 · 新能力

04

Agentic search 的 TTC

算力花在 agent 的循环里，由它自己决定先搜什么、用哪个工具、要不要再来一轮。

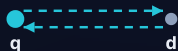
换到的是 · 召回 + 精度



02 | 向量模型的 test-time compute

编码器冻住不动，只在推理期重新组合它已经算出来的东西

这十二个程序，没有一个是人设计的



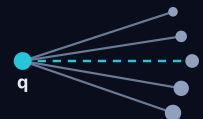
BidirZScore $c=1.2$



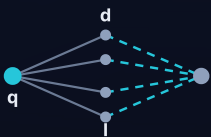
SentMaxSim $c=2.2$



AdaptGranularity $c=2.7$



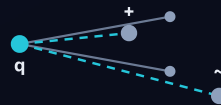
CoverageTriple $c=3.7$



LexicalHybridRRF $c=3.9$



CrossRoundRRF $c=3.9$



DiverseDualCtx $c=5.6$



ConsensusRocchio $c=6.4$



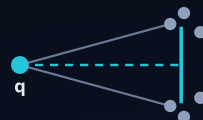
NegContrastive $c=7.2$



MomentumProg $c=9.8$



GraphCentrality $c=12.2$

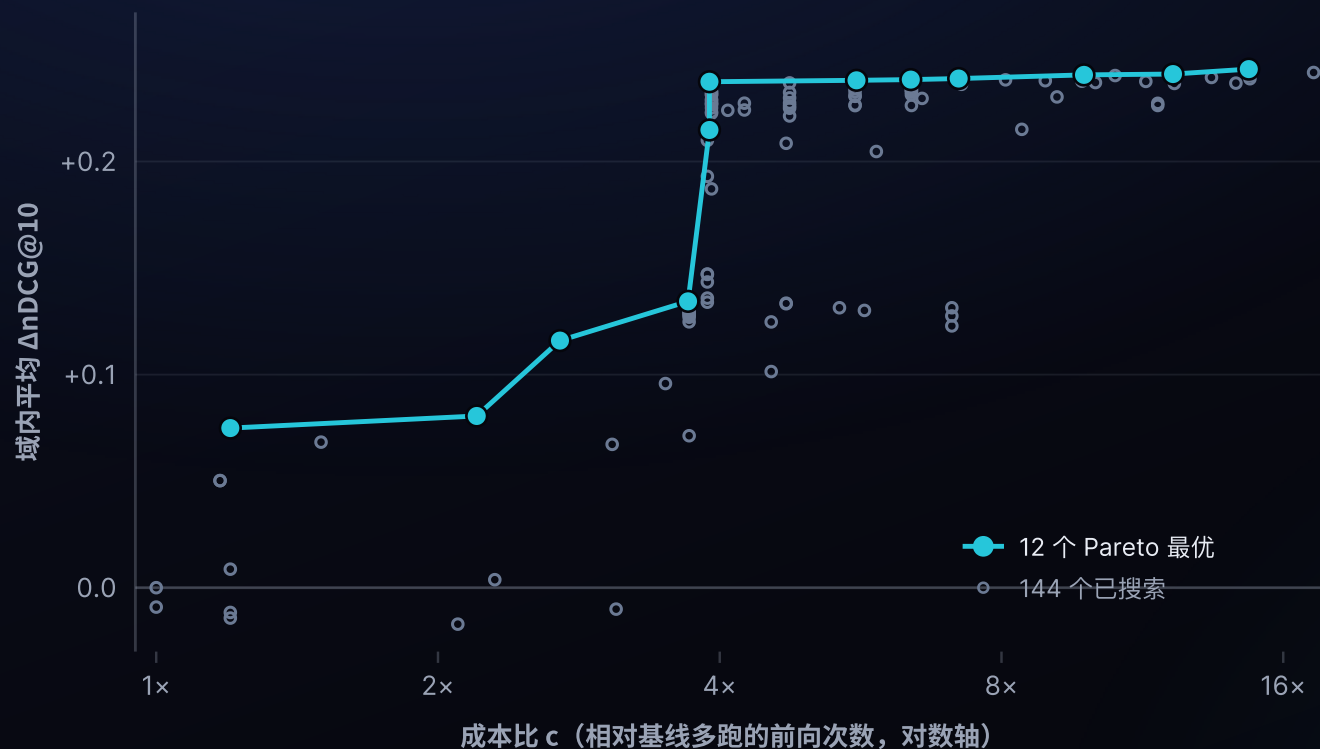


FisherStability $c=14.7$

切句子、z-score 归一、多路融合、质心偏移、构图传播，全部是对同一批冻结向量的重新组合。成本从左往右递增。



每行是十二个程序之一，每列是十九个留出任务之一，四个编码器里三个在发现阶段从未出现。76 个「任务 × 编码器」组合里，有 52 个至少在某个程序下拿到了提升。更有意思的是右边两块：gemma-300m 和 qwen3-0.6b 这两个完全没参与过发现的家族，正格比例反而最高，都是 62%。



144

搜出来的程序总数，全部是对同一批冻结向量的免训练重组

12

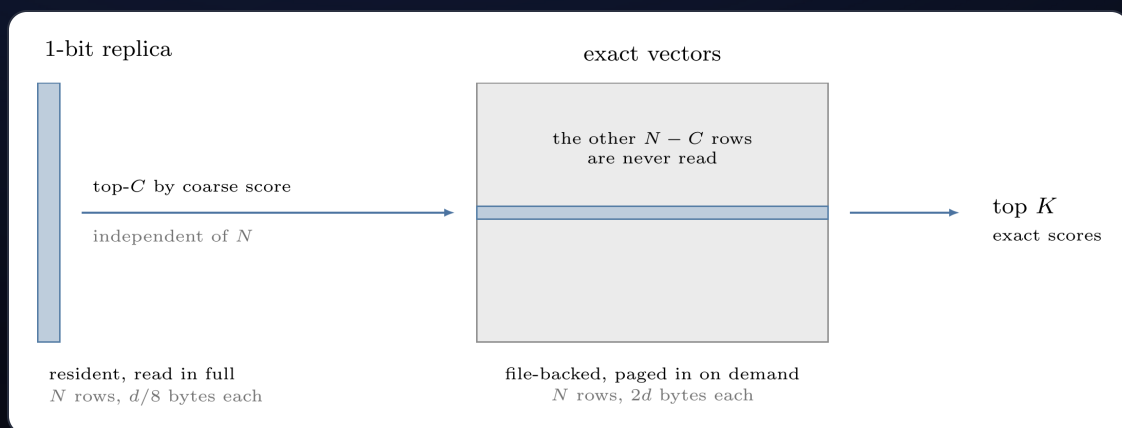
落在 Pareto 前沿上，成本从 1.2 倍排到 14.7 倍

域内平均 $\Delta nDCG@10$ 从 **+0.07** 爬到 **+0.24**。这就是一条标准的 **test-time compute scaling** 曲线，而它发生在一个 239M 的冻结小模型上。



03 | 重排：最熟悉的 test-time scaling

召回给你一堆候选，重排再花一次算力，把它们放在一起比一遍



条宽是每行的字节数，条高是行数：副本只有精确向量的十六分之一宽。青色标出一次查询真正读了什么——副本全读，精确向量只读短名单那一段。

Omni 里没有 ANN 索引，一次查询要扫全库。建一个图索引，光链接就要每行 256 字节，四百万 chunk 时是 0.95 GB，**接近查询要扫的矩阵的三倍**，而且持续编辑的语料会不断让它失效。

所以同一个空间存两份：**扫描读 1 bit 副本，精确向量只有短名单才碰**。粗排分数只决定谁进短名单，不必准，只要短名单够宽；**最终排序由精确重算说了算**。候选集是 $\min(4096, \max(1024, \text{topK} \times 32))$ 。

1.76x

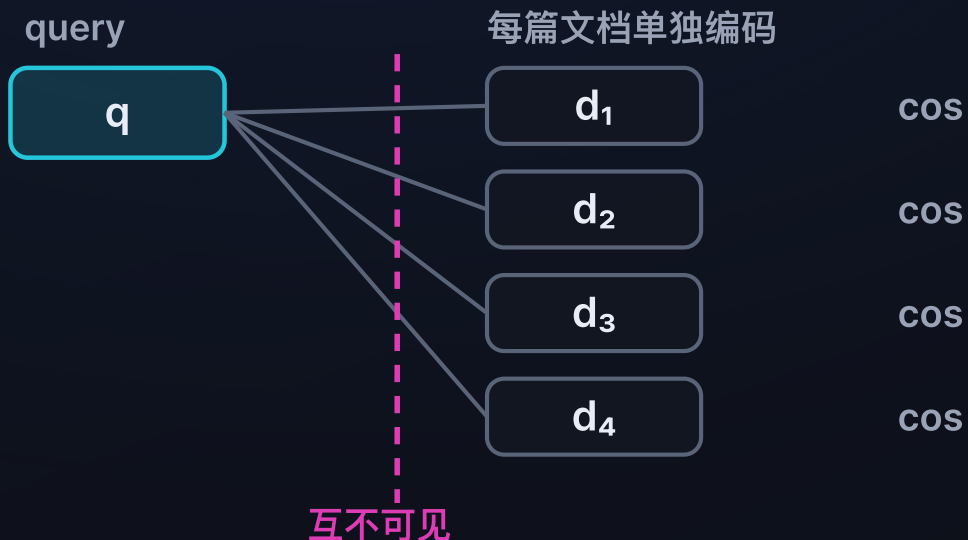
50 万行时相对全量精扫的加速，M3 Pro 14 核

9.7 ms

文本查询 p50，M3 Ultra，真实个人文件库

双塔 bi-encoder

召回



每篇文档的向量，是在别的文档还不存在的时候就算好的。四篇各打各的分，谁也没见过谁。便宜，能扫全库。

listwise 重排

精排



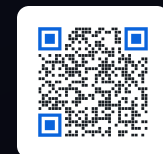
query 和最多 64 篇候选放进同一个上下文，彼此可见。三段都提到延迟时，模型能判断哪一段真的回答了问题。

同样 0.6B 参数，jina-reranker-v3 的 BEIR nDCG@10 是 61.94，而双塔式的 bge-reranker-v2-m3 是 56.51、Qwen3-Reranker-0.6B 是 56.28。差距来自让候选互相可见，不是来自更大的模型。

只有一页文档时，把索引删掉



GITHUB



常规做法是把每个 chunk 都 embed、存向量、再用 ANN 查回来。但索引是靠很多次查询摊薄成本的，而一页文档只活在这一次浏览里。为它建索引，比直接跑一遍前向还贵。

所以这里反过来：**把整页塞进一次 listwise 前向**，答案直接以 <mark> 标回原文，页面自己的 CSS、表格、图都不动。

271 ms

104 句 / 52 chunk / 约 3.5K token，一次请求，M3 Ultra

0

索引、向量库、chunk 数据库，一个都不需要

一百万篇文档仍然需要第一段召回。而只有一页的时候，该删掉的正是召回这一段。

The screenshot displays the jina-reranker-v3.5 web interface. On the left, there's a sidebar with a search input field containing "https://jina.ai/news/jina-reranker-v3.5", a question input field with "how did they make it run faster without making the model bigger", and a ranking section with "granularity" set to "2 sentences" and "highlights" set to "top 1". Below this is a "result" section showing a table with columns "sentences", "chunks", "latency", and "requests". The table shows 104 sentences, 52 chunks, 271 ms latency, and 1 request. A highlighted result shows a score of 0.3134. On the right, the main content area shows a code snippet for reranking documents and a "Conclusion" section. The conclusion states that the practical claim of jina-reranker-v3.5 is narrow and testable, and that it closes most of the gap to a 4B generalist reranker. At the bottom, there are "Categories" and "Featured" buttons.

提问，答案在原页面里点亮。问题里没有一个实词出现在它找到的那句话里。

search_web

引擎自带 snippet

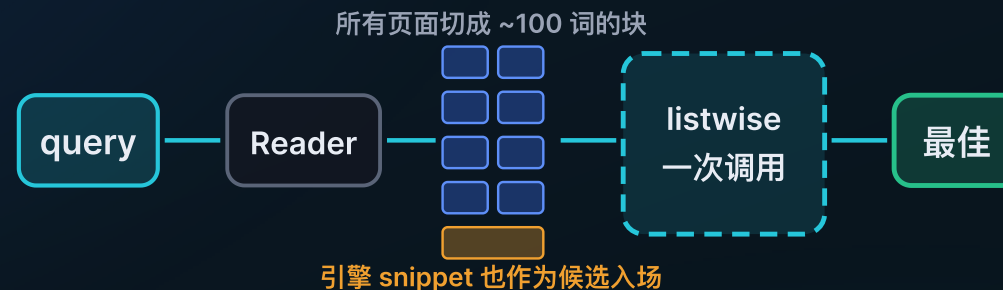


页面开头，或者只是命中了关键词

引擎挑的那一段，通常是页面开头或含关键词的碎片，未必回答了问题。

search_web_deep

正文级 snippet



读正文、切块，所有页面的所有块进同一次 listwise 调用。任何一页的任何一段，都能直接压过另一页的段落。

为什么必须放在一次调用里

listwise 的分数只在同一个 context 内可比。每页各调一次，页内名次可信，跨页排序几乎是任意的：4 个真实 query 实测，逐页排序与合并排序没有一次完全一致，Kendall tau 只有 0.00 到 0.33。合成一次，调用数还从 N+1 降到 1。

让两种 snippet 同场竞争

正文抽取有时反而更差——只抓到小标题，或在 StackOverflow 上选中了提问而不是回答。所以每个 url 同时投入「正文块」和「引擎原始 snippet」两个候选，谁分高用谁。同一次调用里比，所以不需要阈值。

143 ms

53 个块的页面，jina-reranker-v3.5 单次调用中位延迟。v2-base-multilingual 是 4194 ms，**差 29 倍**，塞不进 10 秒预算

72%

两种 snippet 确有差异的 18 组里，正文块胜出的比例

36%

改按固定句数切块时，某中文维基页面被整块丢弃的比例。按大小切，句子覆盖率回到 100%

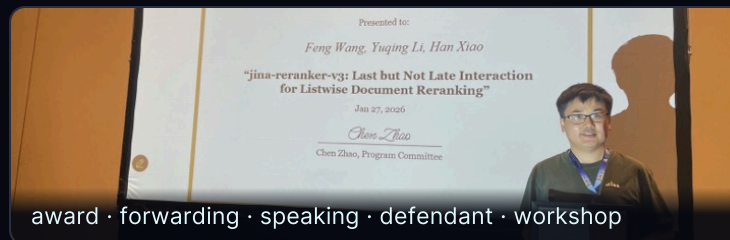
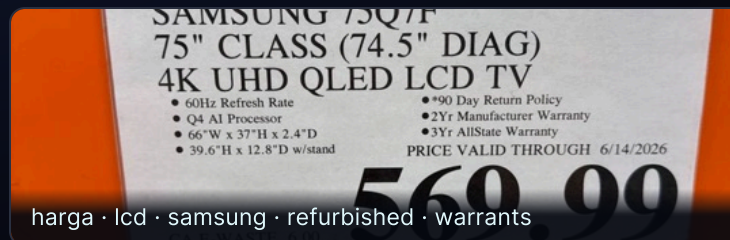
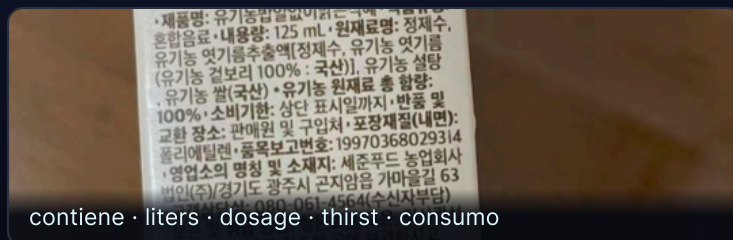
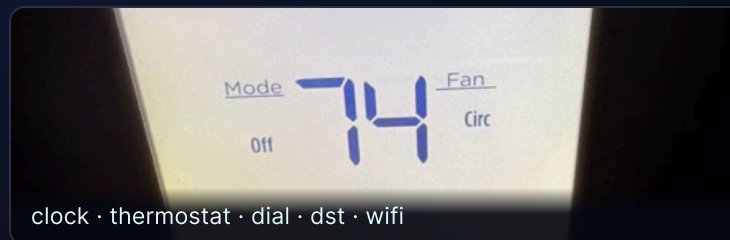
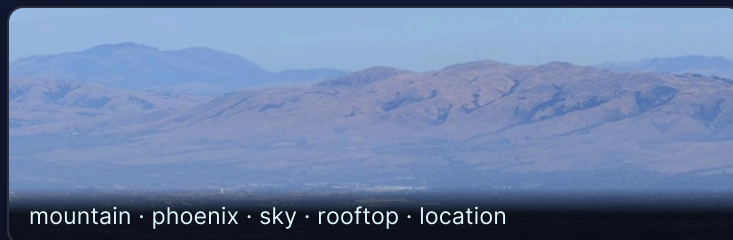
多花的是读正文 + 切块 + 一次 listwise 的钱，换回来的是真正回答了问题的那一段。



04 | 用 test-time compute 换一个新能力

一个只会做检索的模型，从来没学过打标签。这次不求它更准，求它会一件新事

这些标签出自一个检索模型



没训练、没请第二个模型、没查词典，全部由推理期的计算得到，而且天然是多语言的。

- | 手上有 一个 **jina-embeddings-v5-omni-nano**，大约 1B，图和文共用一个 768 维空间。
- | 要它干 把图里**每一个**东西都说出来：多标签，而且词表是开放的。
- | 前提是 权重全冻结。它是个检索编码器，**没有标注头，没有分类器，训练时压根没见过这个任务。**
- | 只准用 它自己吐出来的那些数，在上面做推理期计算。别的一概不行。

不训练

不请第二个模型

不用 WordNet 和词典

不用正则和词性标注

前面几个项目都是把它会的事做得更好，这一次是让它做一件根本不会的事。

A**同一次前向，读得更深**

别只拿那个池化完的向量，把 **patch 级** 的输出全读出来，挨个跟 12.8 万个词打分。
算力几乎没多花。

几乎不额外花钱

B**换个视角，再跑一遍**

把图切成若干 crop **重新编码**。小物体在自己那张 crop 里占满画面，不再被整图的池化冲淡。

算力随视角数线性增长

C**在已有结果上再做文章**

白化、最优传输、图传播、EM——在**同一批像素**上做更花哨的数学。

算力全花在数学上

A 读得更深换来 +0.371 mAP，**B** 看新的像素再换来 +0.075。

离线，只算一次



每张图



每一个方框，不是那个冻结模型，就是普通算术。A 把同一次前向读得更透，B 在新的像素上重新编码。

$$S(\ell) = \underbrace{0.3 \left(\langle g, e_\ell \rangle - \mu_\ell^g \right)}_{\text{全局上下文}} + \underbrace{0.7 \left(\max_{p \in P} \langle p, e_\ell \rangle - \mu_\ell \right)}_{\text{patch 证据}} + \underbrace{1.3 \left(\max_{c=1..14} s_\ell(\text{crop}_c) - \mu_\ell \right)}_{\text{多 crop 重编码}}$$

全局上下文：池化向量减掉自己的背景先验。不额外花钱。

A patch 证据：在前向已经产出的那些行上做代数。不额外花钱，+0.371 mAP。

B 多 crop：14 次新的前向，看新的像素。14 倍成本，+0.075 mAP。

然后 $\ell \in$ 词首过滤后的词表 $\rightarrow$ 向量空间 $\text{NMS}(\tau = 0.6) \rightarrow \text{top-}k$

没有分类头，没有 logits，没有学出来的阈值：三个固定权重、两个背景先验，以及若干次 max。



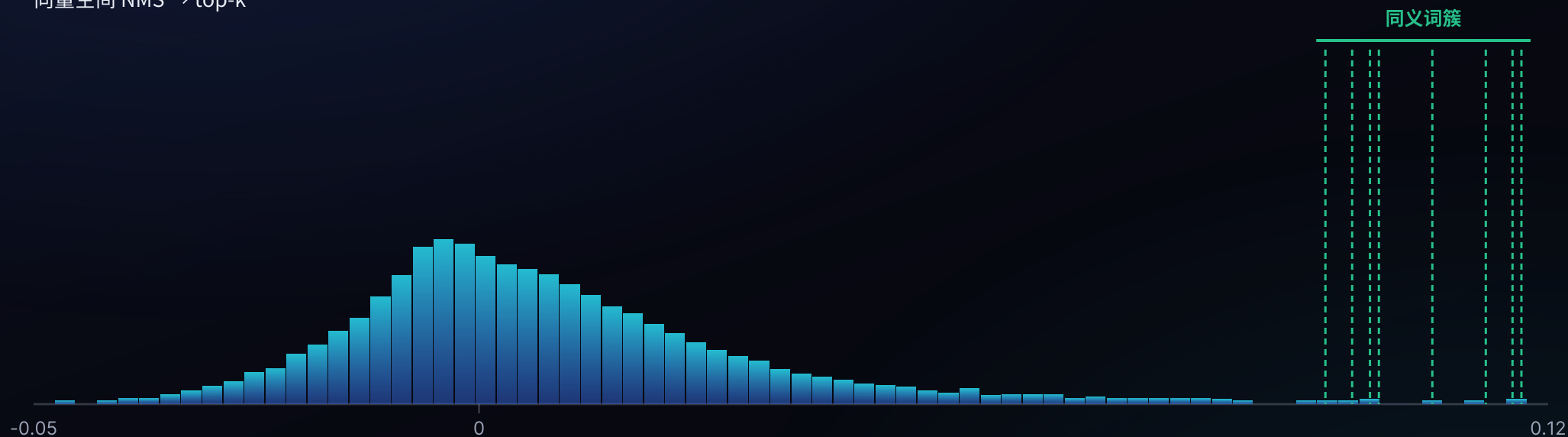
12.8 万个词的得分如何被整理出来



4 · 右尾去重

向量空间 NMS → top-k

token 数 (√ 尺度)



NMS 之前的 top-8: kitty · cat · kitten · chatte · kitt · cats · kittens · cosy, 其实是一个概念

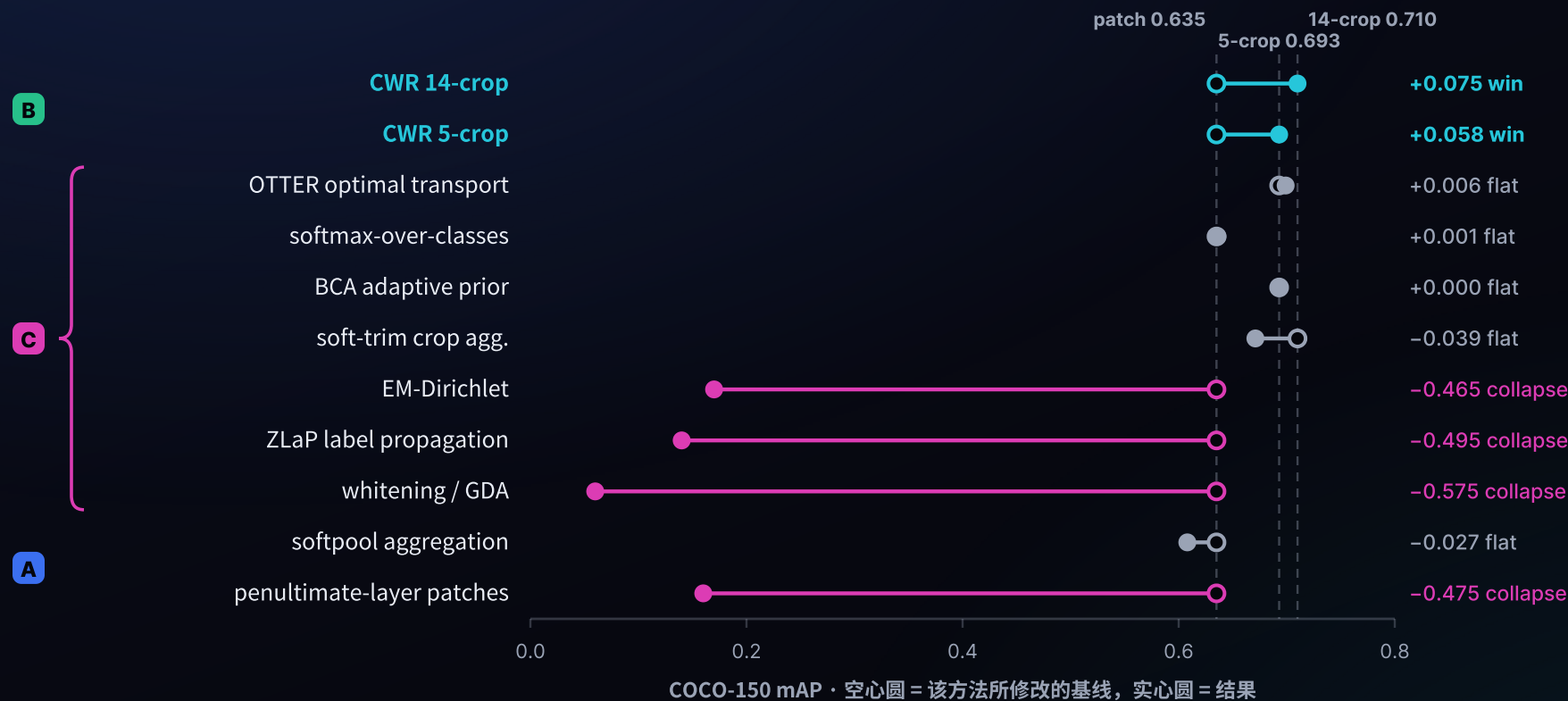
NMS 之后 → kitty · cosy · plush · crib · paw · sleeps



做法	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

只用池化向量是 0.264；改读 patch 之后 0.635；再多切几个 crop 重新编码，0.710。

A 类一步就 +0.371，B 类再补 +0.075。这中间模型一个权重都没动过。



能站住的是 B 类：真的去看新的像素。A 类把这次前向读透。

至于 C 类那些更花哨的数学——它们的前提是图和文在两个空间里，可这儿本来就是一个空间。



能带回新信息的算力才换得到精度



75 毫秒把一次前向读透，1 秒钟看十五个视角。前沿线上的每一步，都让模型读到了上一步没读到的信息。

+1.3 ms

每张图打标签的额外耗时，约为编码成本的 3%，与原本就要跑的前向共用一次计算

0.847

精细模式 (5-crop) P@1, Swift/bf16 端口, 基线 0.773

32 帧

视频每 240 秒采样一次，跨空间和时间一起取 max

图片

建索引时一并产出标签，不需要第二次前向。

视频

抽帧后共用同一套 patch 打分逻辑，时间维并入同一次 max。

扫描版 PDF

当图片处理。OCR 无法识别的扫描件，也能用词检索到。

编码器、索引、存储全在那台存着文件的 Mac 上。从一篇实验记录到一个能用的功能，中间只隔着这 3% 的算力。



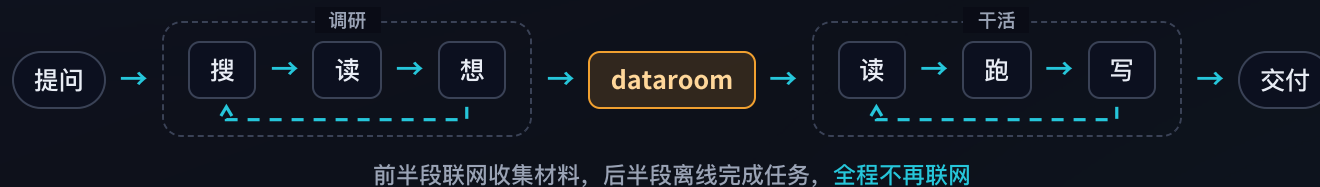
05 | Agentic search 里的 test-time compute

前面两个项目都在模型内部。这一层，算力花在了 agent 的循环里

2025 · Deep research



2026 · 长程任务



调研和执行需要的不是同一种 token。收集材料用便宜的本地模型，昂贵的额度留给真正需要判断力的那一段。

dataroom: 把互联网下成一个 zip



GITHUB



给它一个题目和一个 token 预算，它**搜、读、写**循环推进，把互联网上跟这个题目相关的内容**下成一个带引用的 zip**。

关键是 token 的账：探索阶段用自己部署的 Qwen3.6-35B-A3B、Qwen3.8-27B 跑，把昂贵的前沿额度省下来，留给后面真正需要判断力的那一步。

dataroom

下成语料 (.zip)

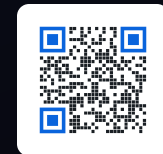


searchbox

只从这个 zip 里找答案



任务实时页面：覆盖度进度、token 消耗、工具调用分布。停止条件是覆盖度达标，而不是预算耗尽。

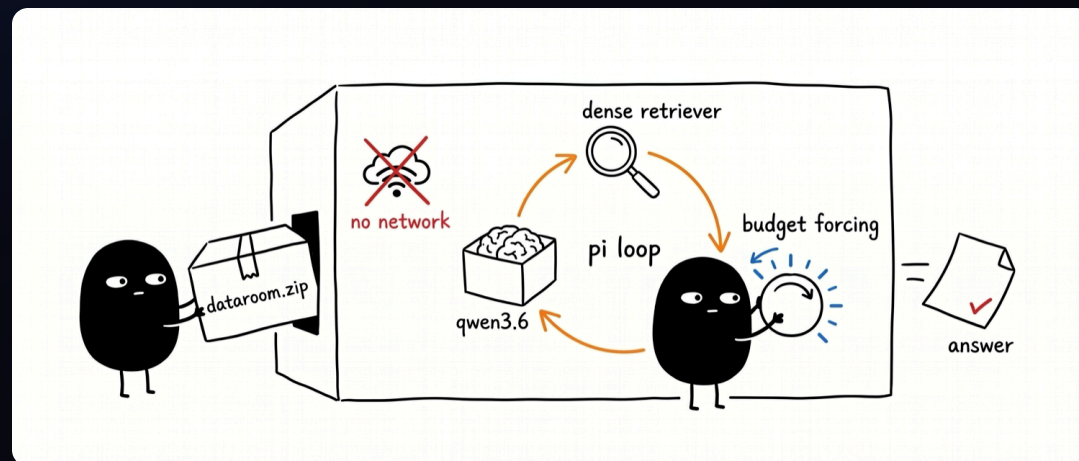


给它一个 dataroom 的 zip，**切断网络**，再问一个问题。自己部署的 Qwen3.6-35B-A3B 在里面跑。它想答上来，只能**用手边的本地工具现搭一条链路**——grep、embed、rerank、相似度、聚类、选多样性。

断网这件事是故意的。**不断网，就分不清它是真的在检索，还是网上碰巧有现成答案。**

正在研究的几个问题

- 它最先调用的是哪个工具？
- 是不是 grep 就够了，稠密检索到底在哪些地方才真有用？
- 提高算力预算，难题上的正确率会不会上升？
- 它搭出来的链路，下次还能不能复用？



断网循环，画成漫画就是这样

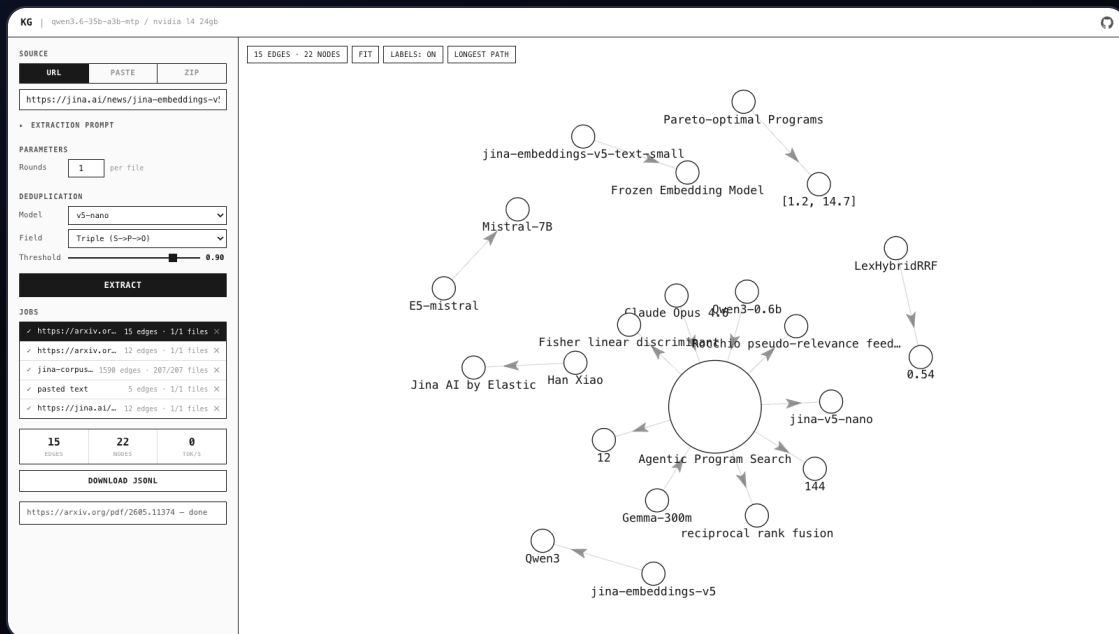


简单题对研究 agent 搜索毫无用处：**一次 grep 就能命中的问题，什么方法做出来都一样分。**要评测 searchbox，得先有真正需要搜索才能答的题。

怎么造

把语料先抽成知识图谱，每条事实是一条 (主语)-[谓语]->(宾语) 的边，然后去**走图里最长的那些路径**。这些长链条自然就成了**多跳问题**——没有任何单独一段文字能直接回答。

题目从同一份语料里长出来，所以是**私有的、不可能被背过的评测集**。agent 必须真的把事实一环一环连起来，也就必须真的去花那份推理算力。



实时图谱：每条事实一条边，最长路径视图会把多跳问题直接挑出来。

前面三个是完整的链路。想知道**检索这一环到底值多少**，得把它单独拎出来：固定一份 Jina 文档语料（210 篇 md 加结构化 json，切成 7777 个 chunk），外面套一个 agent，然后**只给它一个外部工具**。

其余全是 Pi 自带的 read、grep、bash。这样被测的就只有 search_corpus 这一件事，**结果的好坏只能归因于它**。

命中不是答案

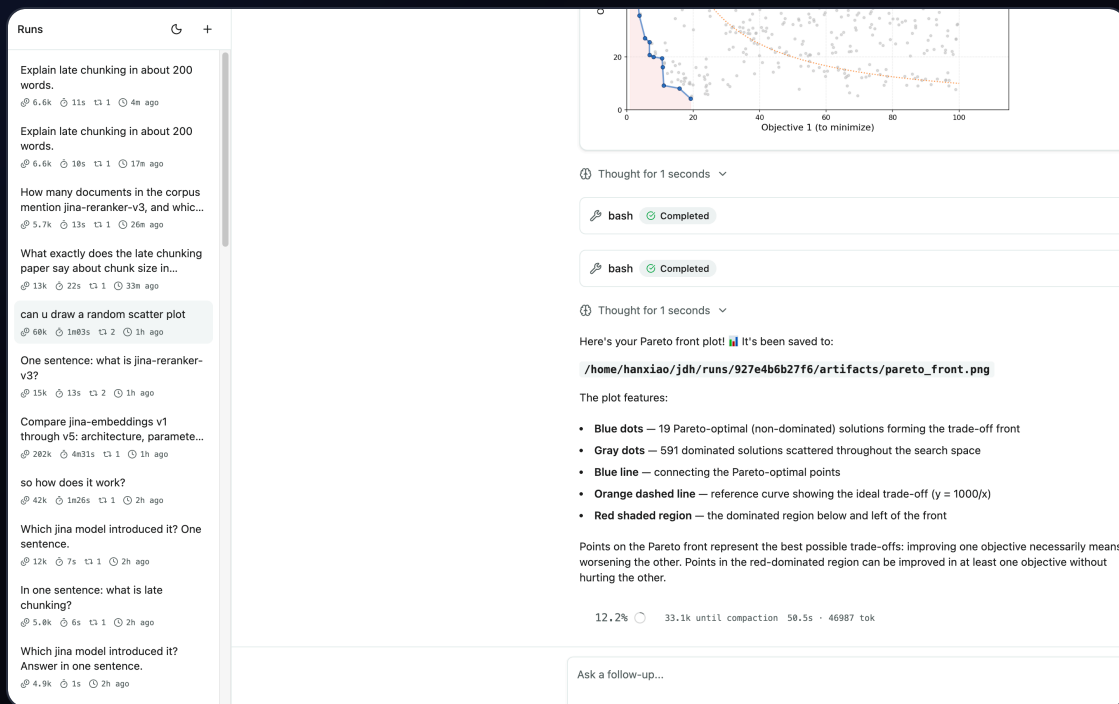
返回 {score, path, lines, text}，agent 得自己回原文把段落读完、再 grep 一遍看还有哪儿提到。

没有 SIDECAR

7777×768 的矩阵直接在扩展里算：加载 117ms，单次检索 4.3ms。少一个需要启动、占用端口、可能失效的服务。

联网是开关

默认关。开了才挂上 search_web 和 read_url，于是「翻书」和「上网」可以分开计分。



跑完一轮的样子：思考流、每一次工具调用及其参数、上下文占用、自动压缩、每段花了多少秒，全部流式开着。

01

dataroom · 把召回做满

便宜的本地 token 把互联网下成一个带引用的 zip。这一步决定了天花板——材料里没有的东西，后面谁也变不出来。

02

searchbox · 把精度做满

断网，只给本地工具，让 agent 在推理期自己组装链路。这一步决定了能从材料里取出多少。

03

知识图谱 · 评测基准

从同一份语料生成多跳难题。前面两步的每一次改动，都靠它来衡量。

04

harness · 单独衡量检索

固定语料，只留一个检索工具。把链路里那一环的贡献单拎出来看。

跟前面几个项目是同一件事，只是换了一层：那边组装的是模型阶段，这边组装的是工具链。都没有把模型换大。



06 | 回到那句话

四块地方，同一笔账

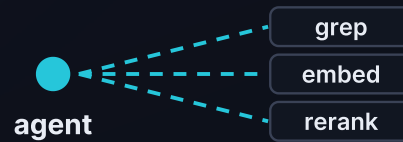


一层在模型里面，一层在模型外面



A 模型内：多算几遍

推理时干的	重组已有向量，加一段模型，或把这次前向读透
项目	向量代数重组 · 重排 · 页内检索 · 图像标注
买到的	精度 + 一个全新的能力



B 模型外：工具链

推理时干的	自己决定先搜什么、再用什么
项目	dataroom · searchbox · 知识图谱 · harness
买到的	召回 + 精度

两层，一个赌注：多花推理算力，而不是换个更大的模型。



01 这次多花的算力，带回了什么新信息？

被池化丢掉的 patch，被缩放丢掉的像素，被单向量抹平的句子结构，被一次 grep 漏掉的那几跳。**说得出带回了什么，这笔算力就花得值。**

02 最便宜的那一档，是不是已经够了？

图像标注里，光是把这次前向读透就吃掉了 +0.371 mAP，之后十五倍的算力只再添 +0.075。**先把这一次前向的输出读完整，通常比再跑十遍更划算。**

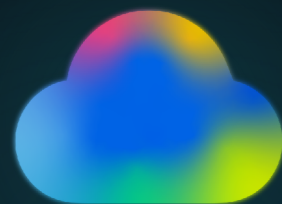
03 这笔算力，该花在链路的哪一段？

一页文档不必建索引，一百万篇仍然需要第一段召回；攒料用便宜的本地 token，判断留给昂贵的模型。**同一笔算力，花在不同环节，回报差很多。**

按这三条走，2026 年的搜索**远远没到没事可做的地步。**



搜索，
就是 **test-time compute**。
高质量搜索，
就是 **test-time scaling**。



THANKS

肖涵 · Elastic 副总裁

微信



IN-PAGE SEARCH

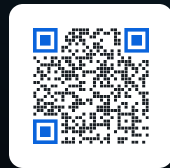
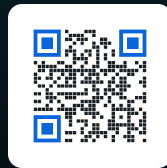


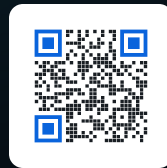
IMAGE TAGGING



DATAROOM



SEARCHBOX



KNOWLEDGE-GRAPH

